PROGRAMMING & CS FUNDAMENTALS

Dynamic Programming Visual Guide

Solve once, reuse forever —
turning exponential recursion into polynomial tables.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Overlapping Subproblems

Naive recursion recomputes the same states exponentially many times.

```
fib(5)

          fib(4)            fib(3)

                fib(3)   fib(2)   fib(2)  fib(1)
```

fib(3) computed TWICE -> fib(2) THREE times...

Call tree has ~2^n nodes for fib(n). The subproblems overlap massively.

- DP applies when: optimal substructure (optimal solution built from optimal subsolutions)...
- ...AND overlapping subproblems (same states recur) — else it's just divide & conquer.
- Two implementations: memoization (top-down) and tabulation (bottom-up). Same complexity.

STEP 2/6

Memoization: Recursion + Memory

Top-down: compute on demand, cache every result.

```
from functools import lru_cache
@lru_cache(maxsize=None)
def fib(n):
    if n < 2: return n
    return fib(n-1) + fib(n-2)   # each n computed ONCE
# O(n) time, O(n) space (cache + recursion stack)
```

- Natural to write: take the naive recursion, add a cache — done.
- Only computes states actually visited — wins on sparse state spaces.
- Costs: recursion depth (stack overflow on deep n) + cache overhead.
- lru_cache turns any pure recursive function into a memoized one in one line.

Memoization answers 'have I seen this state?' Tabulation answers 'in what order do I fill states?' Both kill the recomputation.

STEP 3/6

Tabulation: Bottom-Up Tables

Fill a table in dependency order — no recursion at all.

```
def fib(n):
    dp = [0, 1] + [0]*(n-1)
    for i in range(2, n+1):
        dp[i] = dp[i-1] + dp[i-2]   # subsolutions ready
    return dp[n]

# space-optimized: only dp[i-1], dp[i-2] needed -> O(1) space
a, b = 0, 1
for _ in range(n): a, b = b, a + b
```

- Iterative: no stack overflow, often faster (no call overhead).
- Computes ALL states up to n — wasteful if only a few are needed.
- Space optimization: keep only the previous row(s) — knapsack goes O(n*W) -> O(W).
- Rule: if the transition uses only k previous rows, keep k rows.

STEP 4/6

The DP Recipe

Five steps that solve any DP problem.

1. State

dp[i] = answer for prefix/subproblem i — define it precisely

2. Transition

dp[i] = f(dp[j]) for j < i — how states combine

3. Base cases

dp[0], dp[1]... — the seeds the recurrence grows from

4. Order

fill so dependencies are ready — usually increasing i

5. Answer

which cell holds the final answer? dp[n], max(dp)...

90% of DP debugging is a wrong state definition. If you can't say dp[i] in one sentence, the definition is wrong.

STEP 5/6

Classic Patterns

Three archetypes that cover most DP interviews.

```
# 0/1 KNAPSACK: take-or-skip each item
dp[i][w] = max(dp[i-1][w], dp[i-1][w-wt[i]] + val[i])

# LIS (longest increasing subsequence): O(n^2), or O(n log n) patience
dp[i] = 1 + max(dp[j] for j < i if a[j] < a[i])

# EDIT DISTANCE: insert/delete/replace
dp[i][j] = min(dp[i-1][j]+1, dp[i][j-1]+1, dp[i-1][j-1]+cost)
```

- Knapsack family: subset sum, partition, coin change (unbounded = reuse items).
- Sequence family: LCS, LIS, edit distance — 2D tables over two strings.
- Interval/grid family: matrix paths, burst balloons — expand the interval.

STEP 6/6

DP Traps

Where DP solutions go wrong.

- Greedy confusion: DP explores all choices; greedy commits — coin change needs DP for arbitrary coins.
- State too small: 1D when the transition needs 2D (knapsack needs item index AND weight).
- Wrong order: filling dp[i] before dp[i-1] is ready — tabulation order must respect dependencies.
- Off-by-one bases: dp[0] means 'empty prefix' — get it wrong and everything shifts.
- Exponential state: n=100 with 2^n states isn't DP, it's wishful thinking — find a smaller state.
- Forgetting the space optimization — and forgetting when it breaks reconstruction of the actual solution.

STEP ✓I✓

Key Takeaways

Dynamic Programming — Remember

- DP needs optimal substructure + overlapping subproblems — else it's divide & conquer.
- Memoization (top-down + cache) and tabulation (bottom-up table) give the same complexity.
- Recipe: state -> transition -> base cases -> fill order -> answer cell.
- Knapsack (take-or-skip), LIS/LCS (sequences), edit distance (insert/delete/replace) are the archetypes.
- Optimize space to the last k rows — but keep full tables when you must reconstruct the solution.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev